

SikhSituationBot – Implementation Roadmap

Overview

- **Purpose:** *SikhSituationBot* is envisioned as a chatbot-style web app helping users find guidance from Gurbani (Sikh scripture) for life challenges (stress, anger, fear, etc.). It will tailor responses and explanations for different age groups and levels of spiritual depth, from simple analogies for children to deeper insights for adults.
- **Rationale:** AI chatbots are already used to answer faith-based questions and support emotional well-being, making this a viable approach. We will leverage an existing Gurbani database (such as BaniDB) to ensure authentic scripture content and accurate translations.
- **Approach:** Development proceeds in three stages (PoC, MVP, Production) with increasingly robust features. Each stage builds on the previous one, demonstrating concept feasibility, then adding core functionality, and finally polishing into a scalable application.

Stage 1: Proof of Concept (PoC)

- **Core Features:**
 - **Keyword Search:** User enters a word/topic (e.g. “anger”, “fear”) and the system retrieves matching Gurbani lines (tuks/shabads).
 - **Basic Responses:** Display the original Gurbani text (with transliteration/translation) and a very brief explanation of its relevance in plain language. For example, if the user asks about anger, show a relevant shabad and one or two sentences explaining its message.
 - **Minimal UI:** A simple chat-like interface (web page with input box and message display). No user accounts, no saving of history – just one-off Q&A.
 - **No Personalization:** The PoC just shows that the concept works; it does not track users or adapt over time.
- **Tech & Tools:**
 - **Frontend:** Very basic web page (HTML/CSS/JavaScript) or minimal React/Vue setup to simulate a chat. Use a simple CSS framework (like Bootstrap) for quick styling.
 - **Backend:** Node.js (Express) or Python (Flask) server to handle queries. The logic can be rule-based or use simple text search.
 - **Data:** For PoC, use a small static collection of Gurbani verses (JSON/CSV) or even mock data. Alternatively, call an open API like BaniDB (the Khalis Foundation’s Gurbani API)[3] for real data.
 - **Search Logic:** Straight keyword lookup (e.g. filter by Punjabi or English words). No NLP sophistication yet.

- **Development Tools:** Code editor/IDE (e.g. VS Code with AI-assist like Cursor/Antigravity for rapid coding). Git/GitHub for version control. Simple testing with curl/Postman.
- **Time Estimate: ~2–4 weeks.** This PoC phase is short-term: typically just days or weeks[4]. A small team (2–3 members) working ~5–7 hours per week each could implement this quickly. Focus is on proving we can map keywords to Gurbani verses and return a coherent response.

Stage 2: Minimum Viable Product (MVP)

- **Core Features:**

- **Advanced Search:** Support more flexible queries. Allow natural-language input (e.g. “How can I deal with fear?”) and use keyword expansion or simple NLP to match verses. Consider using embeddings or synonym lists.
- **Tiered Explanations:** For each result, offer multiple explanation versions (e.g. child-friendly, teen-level, adult-level). These can be handcrafted or generated by a model with review. For example, a child version uses simple words and analogies, while an adult version goes into deeper meaning.
- **Improved Chatbot UI:** Develop a proper conversational interface. Implement a message-bubble layout, user avatars, typing indicators, etc. Allow the bot to send suggested follow-up prompts or FAQs.
- **Integrated Gurbani DB:** Replace static data with a real Gurbani source. Connect the backend to a Gurbani API or database (e.g. BaniDB API[3]) so that searches cover the entire Guru Granth Sahib.
- **Content Examples/FAQ:** Create a section of common prompts or FAQs (e.g. dealing with stress, anger) that users can click. Provide example questions to guide new users.
- **Usability Enhancements:** Add basic input validation, response delay for realism, and clear error messages if no result is found.

- **Tech & Tools:**

- **Frontend:** Modern JavaScript framework (React or Vue.js) for a dynamic UI. Use component libraries (Material-UI, Ant Design, or Bootstrap) for faster development.
- **Backend:** Node.js/Express or Python (Flask/Django). Connect to a database (e.g. PostgreSQL, MongoDB) if storing any data (like user feedback or logs). Use the Gurbani API (BaniDB) to fetch verses[3].
- **Search:** Implement a better search engine. Options include ElasticSearch or a vector-search (e.g. Pinecone or Milvus with sentence embeddings) for semantic matching. At minimum, implement fuzzy matching or multi-field queries.
- **NLP/AI:** Integrate a language model (GPT-4/Claude or open-source LLM) to interpret user queries and help formulate explanations. For example, the

model can suggest which verses match the query, and generate age-tailored explanations (under strict supervision to keep fidelity).

- **UI/UX Design:** Use prototyping tools (Figma, Adobe XD) to mock up the chat interface and test user flow before coding.
- **Dev Tools:** Continue using AI-assisted coding tools. Add testing frameworks (Jest/Mocha or pytest) for unit tests. Maintain documentation.
- **Time Estimate: ~3–4 months.** Developing a fully functional MVP (with polished UI, robust search, and multiple explanation modes) is a substantial effort. Industry guidance suggests MVPs take about 3–4 months[5]. For our team, allocate ~10–12 weeks of focused work (with ~5–7 hrs/week per person) for core development and testing. Use Agile sprints: e.g., 2-week sprints with demos.

Stage 3: Production Version

- **Core Features:**

- **AI-Powered Understanding:** Employ advanced NLP to deeply understand user intent. Implement a retriever-augmented generation (RAG) pipeline or fine-tuned LLM that can contextually match life situations to scripture. Ensure the system “understands” synonyms and can handle multi-turn dialogue.
- **Audio Responses:** Add voice output. For users (especially children or the visually impaired), provide audio playback of Gurbani verses and explanations. Use text-to-speech services (AWS Polly, Google Cloud TTS) with appropriate Punjabi and English voices. Possibly include recitations of shabads.
- **User History & Personalization:** Implement user accounts or device-based profiles. Save past queries so the bot can build on earlier sessions. Provide personalized suggestions (e.g. “Last week you asked about stress; here are more passages on peace of mind.”).
- **Responsive Mobile/App Interface:** Deliver a seamless experience on mobile. This could be a Progressive Web App (PWA) that users can install; PWAs “deliver speed, reliability, offline access”[6]. Alternatively, build native apps (React Native/Flutter) with the same backend.
- **Scalability & Reliability:** Host on cloud infrastructure (AWS, GCP, etc.). Use containerization (Docker) and orchestration (Kubernetes) for scaling. Set up monitoring/logging (e.g. with Datadog or CloudWatch). Implement data backups and security (HTTPS, authentication for sensitive APIs).
- **Analytics & Feedback:** Add analytics to track usage patterns and popular queries. Provide a feedback mechanism (e.g. “Was this answer helpful?”) so the system can improve explanations over time.
- **Admin Tools:** (Optional) Create an admin dashboard for content moderators to review and refine bot responses or add new FAQs.

- **Tech & Tools:**

- **Infrastructure:** Cloud services (AWS/GCP/Azure). Utilize serverless functions or container hosts. Set up CI/CD pipelines (GitHub Actions/GitLab CI) to automate testing and deployment.
- **NLP/ML:** Use HuggingFace Transformers or OpenAI API for LLM capabilities. Store embeddings of verses in a vector database (Pinecone, Weaviate) for fast semantic search. Consider caching frequent queries.
- **Frontend (Mobile):** If building a mobile app, use cross-platform tools (React Native or Flutter). If PWA, implement service workers for offline caching (PWA core features[6]).
- **Audio:** Integrate TTS APIs for voice output. Pre-generate or stream audio. For Gurbani recitations, consider using existing shabad audio libraries if licensing permits.
- **Security:** Implement OAuth or similar if accounts are used. Protect API keys and user data.
- **Testing:** Automated unit and integration tests, plus usability testing with real users.
- **Time Estimate: ~6–12 months (or more).** Building a production-grade app with all these features is a major effort. As a guide, comprehensive MVPs can take 6–12 months to develop[5]. We should plan this in phases (start with core new features, then iterate). With part-time student contributions, expect this to stretch into the next term or year.

Team Structure & Commitment

- **Recommended Roles (5–6 members):**
- **Project Lead/Product Manager:** Oversees planning, ensures milestones are met, coordinates team. (5–7 hrs/week)
- **Backend Developer(s):** Builds the API, integrates Gurbani data, implements search/NLP backend. (5–7 hrs/week)
- **Frontend Developer(s)/UI Designer:** Designs and codes the chat interface and mobile views, focuses on UX. (5–7 hrs/week)
- **AI/NLP Specialist:** Handles query interpretation, works on language models, and tunes search relevance. (5–7 hrs/week)
- **Content/Gurbani Expert:** Curates scripture selections, writes and reviews explanations (especially for different age groups), ensures doctrinal accuracy. (5–7 hrs/week)
- **(Optional) DevOps/QA:** Sets up hosting, CI/CD, testing, and ensures reliability.
- **Time Commitment:** Each member should plan **at least 5–7 hours/week**, including time in regular meetings (e.g. weekly stand-ups) and self-study as needed. For example, a 6-member team means ~30–42 person-hours per week on the project. Keeping to this commitment is crucial for on-time completion.

Note to Students: Regular participation is essential. Every team member must contribute consistently each week (development, design, content writing, testing, etc.). Falling behind or skipping contributions will delay progress for everyone. Stay engaged: attend all team meetings, update tasks promptly, and communicate blockers early. Only through steady, collaborative effort can this roadmap be achieved.

Sources: We will leverage the BaniDB Gurbani API for scripture data and build on insights about AI chatbot development and timelines. Each stage's timeline is a rough estimate based on industry guidance for PoCs, MVPs, and full releases.